

# Build an extendable agent harness for enterprise work

**Time box:** 3 calendar days. The required parts are sized for roughly 10–14 hours of focused work; with optional parts it's closer to 18–24. Do the required parts well, then pick what to add. Tell us what you cut and why — that's part of the answer.

---

## 1. The problem

We're building a platform where every employee at a manufacturer gets a personal AI agent. The agent understands their role and current work, reasons across ERP data, email, calendar, and internal knowledge, notices things that need attention before being asked, and recommends or executes actions inside company systems — within that person's permissions, with human approval where it matters, and with a full explanation of why.

## 2. The scenario you must make work (Scenario A)

A purchasing manager's agent should detect and handle this:

A critical part is projected to run out in five days. The open PO for it is with Supplier Y. An email from Supplier Y says the shipment slips to next Tuesday. Production order 4812 consumes the part and starts before then. The agent alerts the manager:

*"Part X will likely cause production order 4812 to miss its scheduled start. Supplier Y said the shipment is delayed until Tuesday. I can move the PO to Supplier Z and notify production. Want me to proceed?"*

On approval, the agent creates the replacement PO, cancels or reduces the original, notifies production, and schedules a check for Tuesday to confirm the new shipment actually arrived. The manager's calendar shows them out of office tomorrow. Rule: if an approval request is unanswered at end of day *and* the approver's calendar shows them out the next day, it routes to their designated backup.

Your harness must:

1. **Detect** the condition without being prompted (a detector runs on a schedule or on an ERP change event).
2. **Gather context** from ERP, inbox, and calendar via distinct providers, scoped to what this user can see.
3. **Reason** to a recommendation and a proposed action plan.
4. **Gate** the plan: check permissions, apply policy (PO value thresholds, backup approver from calendar), and require human approval before any write.
5. **Execute** the approved plan through tools you define, idempotently, with each step and its rationale logged.
6. **Follow up**: create a scheduled task that re-checks on Tuesday and re-enters the loop if the shipment is still missing.
7. **Explain**: from the audit log alone, someone should be able to reconstruct what the agent saw, what it concluded, what it was allowed to do, who approved what, and what actually happened in each system.

### 3. What you model

You build the company. At minimum:

- **Systems and data.** An ERP (parts, suppliers, purchase orders, production orders — and whatever else the scenarios need), an inbox, and a calendar. Static files, SQLite, an in-process fake API — your choice. Include noise: irrelevant emails, unrelated POs, a supplier that looks attractive but shouldn't be used.
- **A permission model.** Users with roles, per-system scopes for reads and writes, an approval threshold (e.g. PO value above which a manager must approve), and a backup approver. Providers must not return data a user can't read; tools must not run without the user's write scope.
- **A tool catalog.** The actions your agent can take, each with a schema, required scopes, and whatever your harness needs to run it safely.
- **A clock.** The scenario is relative to "today." Your demo has to show the Tuesday follow-up firing, so time must be something you can advance.

Keep it as small as the scenarios allow — the sample schemas in the appendix are about the right size. We'd rather see a handful of entities and four tools that make the story real than a fake ERP with forty tables. Write a short **MODEL.md** explaining what you kept from the sample, what you changed or added, what you left out, and why.

Use a real LLM API of your choice.

### 4. Part 1 (required) — the harness and Scenario A

Build the harness and run Scenario A end to end. The responsibilities below need to exist somewhere and be separately replaceable. How you factor them is your call — we'd rather see a decomposition you can defend than a module per bullet:

- detecting attention items on a schedule or event
- gathering context per system, scoped to the user
- planning: attention item + context → proposed actions, with reasoning
- gating: permissions, policy, approval routing — enforced in code
- executing actions with idempotency and compensation
- memory: what the current run knows vs. what persists across runs
- scheduling deferred work that survives a restart
- an append-only audit of every step, input, decision, approval, and outcome

## 5. Part 2 (required) — deterministic workflows

Purchasing has told us:

"We don't want the AI improvising a PO reroute. The steps are fixed: confirm the alternate supplier is approved for the part → confirm their lead time meets the production date → create the new PO → cancel or reduce the old one → notify production → schedule the arrival check. In that order. Every time."

So the reroute in Scenario A must execute as a **declared workflow**, not as a free-form plan the model assembled. The free-form planner still exists — it decides that a reroute is the right response and supplies the parameters — but once it enters the workflow, the definition is in charge. (Scenario B, below, has no workflow and runs through the free-form path, so both paths stay exercised.)

Requirements:

- Step order is fixed by the definition, not by the model.
- The agent decides *whether* to enter the workflow and supplies its parameters. It does not reorder, skip, or add steps.
- LLM steps inside the workflow are allowed but bounded: e.g. choose among pre-filtered approved suppliers with a stated justification; draft the notification text. Show how you constrain what the model can do in those steps.
- Each step is idempotent and has a compensation (what to undo if a later step fails).
- State is persisted after each step; a killed process can resume where it left off.
- Definitions carry a version. Handling in-flight instances across a version change is a design-doc question, not something you need to implement.

In your design doc, answer: if you were designing a deterministic workflow engine from the start, where reasoning is just a small node inside the graph rather than the thing that drives it, would you still build it the way you did here? If not, how would you design it differently?

## 6. Part 3 (Scenario B)

Add this scenario **after** Parts 1 and 2 are done:

A lot of a lot-tracked part has been placed on quality hold. A production order is allocated to consume that lot in three days. The quality manager's agent should notice, find whether another good lot of the same part can cover it, and propose: reallocate the lot and notify the production supervisor — or, if no lot covers it, flag a shortage to purchasing.

This needs whatever your model didn't have yet — quality/lot data, a new detector, a new context provider or extension of one, at least one new tool, and a different user with different scopes. If Scenario B required editing the planner, the gate, or the audit layer, tell us why.

## 8. Design doc (2–3 pages)

Short, written answers for the parts you didn't build. The first three are required; the rest are optional.

- **(required) Identity and authorization:** how a real deployment establishes who the user is (SSO, token exchange) and how per-system permissions flow through to the tool layer without the agent holding standing credentials.
- **(required) Long-term memory:** what gets promoted from a run into durable context, how it's kept accurate, and how you avoid the agent acting on stale beliefs.
- **(required) Scaling to thousands of employees:** where your current design breaks first, and what you'd change.
- **Connecting real systems:** how your provider and tool abstractions map onto a real ERP API, Microsoft Graph, and a document store. What changes; what doesn't.
- **Observability and evaluation:** what you'd trace, what "the agent did a good job" means measurably, and how you'd catch a regression in recommendation quality before users do.

## 9. Deliverables

- A repo that runs with one documented command: Scenario A through approval and execution, the clock advanced to Tuesday with the follow-up firing, Scenario B, then your failure cases.
- **MODEL.md:** what you modeled and why.
- **README:** how to run, how to add a tool / provider / detector / workflow, what you cut.
- The design doc.
- Tests covering the gate, trigger dedupe, and workflow resumption at minimum.
- A recorded run (log or transcript) of Scenario A showing the approval prompt, the execution, and the audit trail.

No UI is required. A CLI or HTTP endpoint for the approval step is fine.

---

## Appendix — sample connector schemas

A starting point, not a requirement. Rename, reshape, add, or drop fields as your model needs. One example record per entity.

### ERP

JSON

```
// parts
{ "part_id": "P-4471", "description": "Stepper motor, NEMA 23,
2.8A",
  "on_hand": 150, "daily_usage": 30, "safety_stock": 20,
  "unit_cost": 42.00, "lot_tracked": false }

// suppliers
{ "supplier_id": "S-Z", "name": "Meridian Drives", "contact_email":
"sales@meridiandrives.example",
  "approved": true, "approved_parts": ["P-4471"], "lead_time_days":
2, "pricing": { "P-4471": 46.50 } }

// purchase_orders
{ "po_id": "P0-77812", "part_id": "P-4471", "supplier_id": "S-Y",
  "qty": 400, "unit_price": 42.00,
  "total_value": 16800.00, "ordered_date": "2026-08-26",
  "promised_date": "2026-09-04",
  "status": "open", "created_by": "u-101" }

// production_orders
{ "prod_order_id": "4812", "product": "PRD-CX200 Conveyor Drive
Unit", "qty": 10,
  "scheduled_start": "2026-09-07", "scheduled_end": "2026-09-10",
  "status": "planned",
  "line": "Line 2", "supervisor_id": "u-301",
  "components": [ { "part_id": "P-4471", "qty": 120 }, { "part_id":
"P-2210", "qty": 30 } ] }
```

```
// quality_lots
{ "lot_id": "L-2093", "part_id": "P-1180", "qty": 100, "status":
"hold", "received_date": "2026-08-28",
  "allocated_to": ["4820"], "hold_reason": "Surface finish 3.4 Ra
vs spec 3.2 Ra",
  "hold_placed_by": "u-202", "hold_placed_on": "2026-09-02" }
```

## Mail

```
JSON
{ "message_id": "M-001", "from":
"rita.alvarez@kestrelcomponents.example",
  "to": ["dana.whitfield@northfield-mfg.example"], "date":
"2026-09-01T16:42:00",
  "subject": "Re: PO-77812 - shipment update",
  "body": "Revised ship date is Monday 9/7, which puts it on your
dock Tuesday 9/8." }
```

## Calendar

```
JSON
{ "event_id": "E-002", "owner": "u-101", "start":
"2026-09-03T00:00:00", "end": "2026-09-04T23:59:59",
  "title": "Out of office - supplier site visit", "attendees": [],
"out_of_office": true }
```

## Users and permissions

```
JSON
{ "user_id": "u-101", "name": "Dana Whitfield", "role": "Purchasing
Manager",
  "manager_id": "u-100", "backup_approver_id": "u-102",
  "scopes": ["erp:po:read", "erp:po:create", "erp:po:cancel",
"erp:production:read",
```

```
        "mail:read", "mail:send", "calendar:read",  
"production:notify"],  
    "approval_limits": { "po_create_max_value": 25000 } }
```

## Clock

JSON

```
{ "today": "2026-09-02" }
```